The Math of Color

How a computer turns a picture into a palette —
and proves you can actually read it

> **What you need to know already:** how to plot a point like (3, 5) on a graph, the Pythagoras theorem, what a square root and a power are, and what the vertex of a parabola is.
>
> **What you don't need:** calculus, matrices, or any prior color theory. Every symbol is explained the first time it shows up.

Companion to the *Rangoli / ACCORD* project plan · Technical math reference, written for a first-time reader

What's inside

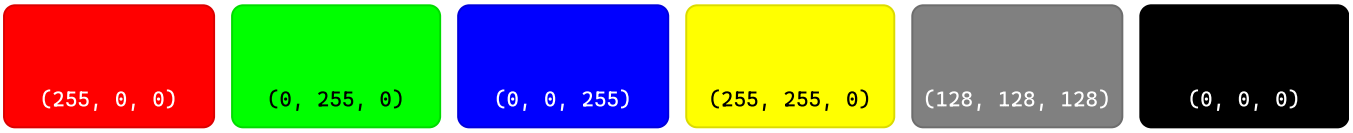
1 A color is just three numbers

Before any math, we need to agree on what a color *is* to a computer.

Zoom far enough into any screen and you find tiny lamps in groups of three: one red, one green, one blue. Every color you have ever seen on a screen is made by setting the brightness of those three lamps. Nothing else is happening.

Each lamp gets a number from **0** (off) to **255** (fully on). So a color is an ordered triple:

```
color = (R, G, B)
```

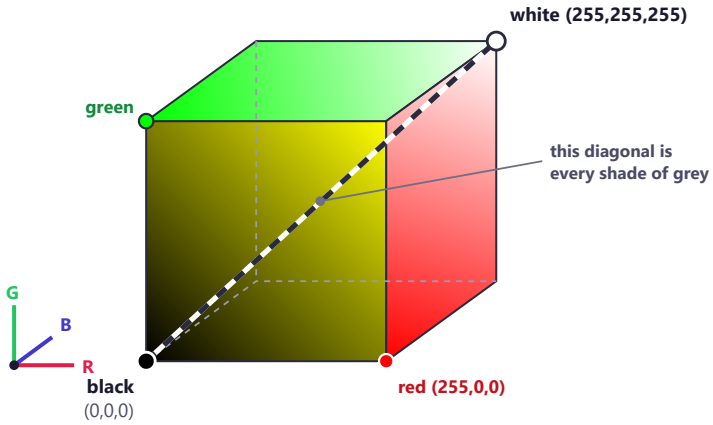


Why 0–255? Because that's exactly what fits in one byte (8 bits, so 2^8 = 256 possible values). Three bytes per pixel, 256 × 256 × 256 = **16,777,216** possible colors. That's where "16.7 million colors" on a monitor box comes from.

This is secretly a coordinate system

Here is the move that turns color into mathematics. Those three numbers are exactly like the (x, y, z) coordinates of a point in space. So:

- Red is the x-axis, green is the y-axis, blue is the z-axis.
- Every possible color is one point inside a cube that runs from 0 to 255 on each side.
- Black (0,0,0) is one corner. White (255,255,255) is the opposite corner.
- The straight line joining those two corners is every shade of grey.



The RGB cube. Every screen color is a point inside it. This picture is the reason we can use geometry on color at all — but as the next section shows, it is a distorted map.

THE KEY IDEA

Once colors are points in space, "how different are these two colors?" becomes "how far apart are these two points?" — and that is a question mathematics can answer exactly. The entire project rests on this one translation.

2 The problem: RGB lies about distance

The cube is a map of color. But it's a bad map — distances on it don't match what your eyes report.

Let's test it. Below are two pairs of colors. In **both** pairs, the two colors are exactly **40 units apart** in the green channel — the same mathematical distance.

Pair A — two greens, 40 apart:

`(0, 120, 0)` `(0, 160, 0)`

Pair B — two blues, also 40 apart:

`(0, 0, 120)` `(0, 40, 120)`

Look at them. The greens in Pair A are clearly different shades. The blues in Pair B are much harder to tell apart. Yet the mathematics insists they are *equally* different, because in both cases one number changed by 40.

WHY THIS IS A REAL PROBLEM

Any algorithm that groups similar colors together has to ask "are these two close?" thousands of times. If the ruler it uses gives the wrong answer, everything built on top is wrong. A palette generator using raw RGB distance will happily merge two blues a human sees as distinct, while splitting two greens a human sees as the same.

The technical name for the flaw

RGB is **not perceptually uniform**. That phrase means exactly: *equal steps in the numbers do not produce equal steps in what you see.*

There are two separate reasons, and we fix them one at a time:

REASON 1

Gamma

The stored number isn't the actual amount of light. Fixed in Section 3.

REASON 2

Eye response

Your eye is far more sensitive to green than blue. Fixed in Sections 4–5.

3 Repair #1 — undo the screen's cheat

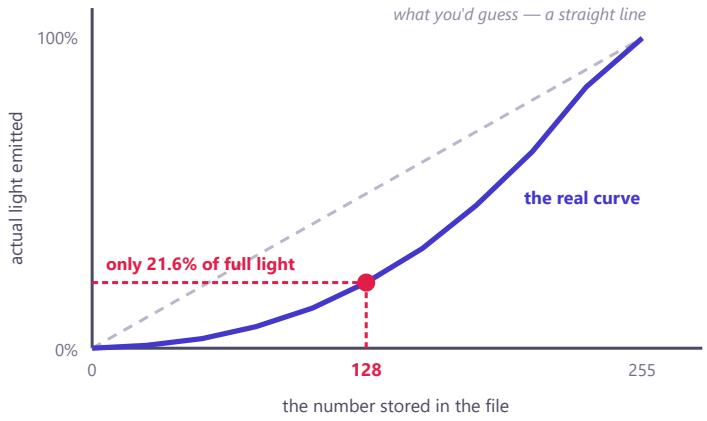
The single most surprising fact in this document: the number 128 does not mean half brightness. It means about 21.6% brightness.

Why screens cheat

Human vision is **much** better at telling dark shades apart than bright ones. Put a candle in a dark room and it's dramatic. Add one candle to a room with a hundred candles and nobody notices.

Now imagine you're designing image storage and you only get 256 levels per channel. If you spread those 256 levels evenly across physical light output, you'd waste most of them at the bright end where nobody can see the difference — and you'd have far too few in the dark end, where every step is obvious. Dark areas would show ugly visible bands.

So the designers of sRGB (the standard color format of the internet) did something clever: they **packed more of the 256 levels into the dark region**, where our eyes need them. The stored number is therefore not "how much light" — it's closer to "how bright this looks." The relationship is a curve, not a straight line.



The gamma curve. Because it bends downward, the middle stored value (128) produces far less than half the physical light. Every step near 0 is a tiny change in light; every step near 255 is a big one.

The formula that undoes it

To do honest physics on light, we have to convert the stored numbers back into real light amounts. This is called **linearizing**. First scale the 0–255 value down to 0–1 by dividing by 255. Then apply:

$$C_{linear} = C \,/\, 12.92 \qquad \text{if } C \le 0.04045$$

$$C_{linear} = \left((C + 0.055) \,/\, 1.055 \right)^{2.4} \qquad \text{otherwise}$$

Two branches, because the curve is a straight line very near zero and a power curve everywhere else. (The straight bit avoids a mathematical problem right at black; ignore it for now, we'll meet the same trick again in Section 5.)

WORK IT OUT YOURSELF

Take the middle value, 128.

`128 ÷ 255 = 0.502` → that's above 0.04045, so use the second branch.

`(0.502 + 0.055) ÷ 1.055 = 0.528`

$0.528^{2.4} = 0.216$

So RGB 128 emits **21.6%** of full light — not 50%. Nearly every bug in amateur color code traces back to someone forgetting this step.

4 Repair #2 — measure like an eye, not a camera

Now we have honest light amounts. But "how much red light" still isn't the same as "what your eye reports."

Inside your retina are three kinds of cone cells, each sensitive to a different range of wavelengths. They're usually called L, M and S (long, medium, short). Crucially, they **don't** line up with a screen's red, green and blue lamps — their sensitivity ranges overlap heavily, and they are wildly unequal in strength.

In the 1920s the CIE (the international lighting standards body) ran experiments where people matched colors by mixing lights, and from that data they built a standard system called **XYZ**. Think of X, Y and Z as "what a standard human eye actually measures."

Converting is just three weighted recipes

```
X = 0.4124 R + 0.3576 G + 0.1804 B
Y = 0.2127 R + 0.7152 G + 0.0722 B
Z = 0.0193 R + 0.1192 G + 0.9503 B

R, G, B here must be the LINEAR values from Section 3
```

That's it — three ordinary weighted sums. (If you've met matrices, this is one 3×3 matrix multiplication. If you haven't, nothing is lost: it's three lines of "multiply and add.")

Look closely at the middle row

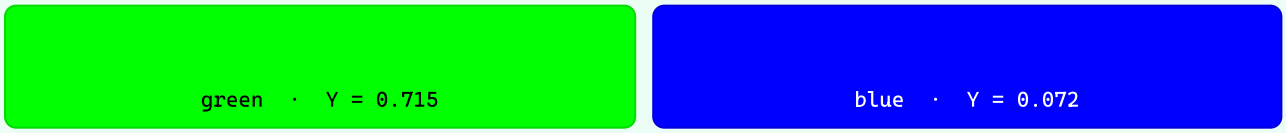
The Y row is special. Y is **luminance** — how bright a color appears. And look at those weights:

RED CONTRIBUTES	GREEN CONTRIBUTES	BLUE CONTRIBUTES
21.3%	71.5%	7.2%

Green does almost three-quarters of the work. Blue does almost nothing. This isn't an arbitrary choice — it's measured biology. Your eye has far more green-sensitive machinery than blue-sensitive machinery.

SEE IT FOR YOURSELF

These two swatches are both "full blast" on one channel — (0,255,0) and (0,0,255). Mathematically symmetric. Perceptually not even close.



Pure green is about **10× brighter** to your eye than pure blue. This single fact explains why yellow highlighter works and blue highlighter doesn't, and why blue text on black is so hard to read.

5 Repair #3 — the honest map (CIELAB)

One flaw left. Y is physically correct, but your *brain* doesn't respond to light in a straight line.

Double the physical light and it does not look twice as bright — it looks maybe 25% brighter. Psychologists found that perceived intensity follows roughly a **cube root** of physical intensity. So to build a scale where equal steps *look* equal, we take cube roots.

That is the core of **CIELAB** (usually just "Lab"), defined by the CIE in 1976 and still the workhorse of color science.

The formula

First, divide X, Y, Z by the values for reference white (the brightest white the system can show): $X_n = 0.9505$, $Y_n = 1.0000$, $Z_n = 1.0888$. Then apply this helper function to each:

```
f(t) = t^(1/3)                          if t is bigger than 0.008856
f(t) = (903.3 × t + 16) / 116           otherwise (the straight-line bit near black)
```

And finally:

$$L^* = 116 \times f(Y/Y_n) - 16$$

$$a^* = 500 \times \left(f(X/X_n) - f(Y/Y_n) \right)$$

$$b^* = 200 \times \left(f(Y/Y_n) - f(Z/Z_n) \right)$$

WHERE 116 AND 16 COME FROM — THEY'RE NOT MAGIC

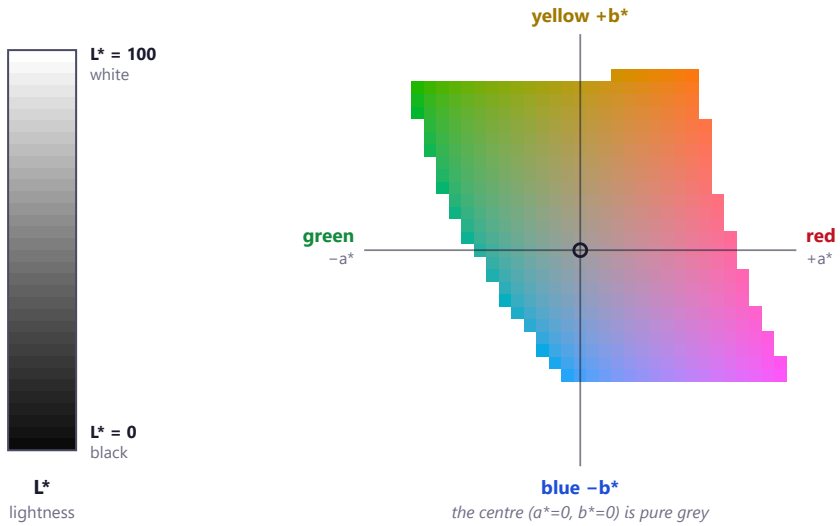
The designers wanted L* to run from exactly 0 (black) to exactly 100 (white). Check it:

Pure white: $Y/Y_n = 1$, and $1^{1/3} = 1$. So $L^* = 116 \times 1 - 16 =$ **100**. ✓

Pure black: $Y/Y_n = 0$, so we use the straight-line branch: $f(0) = (0 + 16)/116 = 16/116$. So $L^* = 116 \times (16/116) - 16 = 16 - 16 =$ **0**. ✓

The 116 and the 16 exist purely to pin the ends of the scale at 0 and 100. Once you see it, the formula stops looking arbitrary.

What the three numbers mean



*The CIELAB space, drawn from real numbers. **Left:** the L* axis, black to white. **Right:** an actual horizontal slice through the space at L* = 65 — every patch is a genuine Lab color converted back to sRGB. The ragged edge is where colors stop being displayable on a screen.*

WHY GREEN–RED AND BLUE–YELLOW, OF ALL PAIRINGS?

Try to imagine a color that is reddish-green. Or bluish-yellow. You can't — and neither can anyone else. These are genuinely impossible colors.

That's because your brain doesn't store color as "amount of red, amount of green, amount of blue." It stores it as two *seesaws*: one that tips between red and green, one that tips between blue and yellow. A seesaw can't tip both ways at once, which is why those colors can't exist. This is Hering's **opponent process theory**, from the 1890s.

a* and b* are exactly those two seesaws. Lab isn't just a mathematical convenience — it's shaped like your visual system.

Where we've arrived

Space	What it represents	Equal steps look equal?
sRGB	Screen lamp settings	**No** — badly distorted
Linear RGB	Actual light energy	**No** — physics, not perception
XYZ	What a standard eye measures	**Better**, still not uniform
CIELAB	What the brain perceives	**Yes** — close enough to build on

Every color in the picture now sits at an honest (L*, a*, b*) coordinate. We can finally measure distance.

6 Measuring distance: Pythagoras in 3D

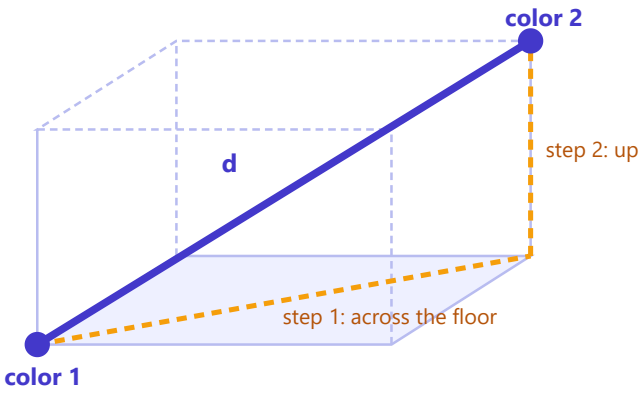
You already know this formula. It just gets one more term.

On a flat page, the distance between (x_1, y_1) and (x_2, y_2) is Pythagoras:

$$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2}$$

In 3D, you apply Pythagoras twice — once along the floor of a box, then once up the vertical edge — and the two square roots collapse into one. You just add the third squared term:

$$d = \sqrt{(x_2-x_1)^2 + (y_2-y_1)^2 + (z_2-z_1)^2}$$



Pythagoras twice. The formula for the long blue diagonal is the flat-page formula with one extra squared term.

Applied to Lab, this gives the original 1976 color-difference formula, called ΔE_{76}. (Δ is the Greek letter delta, used throughout science to mean "change in" or "difference in." E is from the German *Empfindung*, "sensation.")

$$\Delta E_{76} = \sqrt{\Delta L^{*2} + \Delta a^{*2} + \Delta b^{*2}}$$

And the number it produces has a real-world meaning, which is the whole point of all that work:

ΔE	What it means to a human
0	Identical colors
~1	The **just-noticeable difference** — a trained eye, colors side by side, can just barely tell
2–3	Noticeable if you look carefully
5	Clearly two different colors
10+	Obviously, unmistakably different

7 Bending the ruler: ΔE_{00}

Lab got us most of the way. But when scientists tested it carefully against real human judgments, small errors remained — and one big one.

Lab is *nearly* perceptually uniform, not perfectly. Three residual problems:

- We're pickier about differences between **dull, greyish colors** than between **vivid, saturated** ones. Two nearly-identical greys look different; two nearly-identical bright crimsons look the same.
- We're less sensitive to lightness changes at the **very dark and very bright ends** than in the middle.
- **Blue is a mess.** The blue-violet region of Lab is measurably more warped than anywhere else.

In 2001 the CIE published a patch: **CIEDE2000**, written ΔE_{00}. It looks terrifying, but it is doing something simple.

ONE SENTENCE THAT DECODES THE WHOLE FORMULA

ΔE_{00} is the ordinary 3D distance formula with each term **divided by a correction factor** — plus one extra term that fixes blue.

Dividing by a big number makes a difference *count for less*. It's grading on a curve: the same 5-point gap means less on an easy test than on a hard one. Each correction factor asks "how hard is this comparison for a human?" and scales the score accordingly.

The shape of it

First, Lab's a* and b* get rewritten as **chroma** (C, meaning colorfulness — the distance out from the grey centre line) and **hue** (H, the angle around it). Then:

$$\Delta E_{00} = \sqrt{\left(\frac{\Delta L}{S_L}\right)^2 + \left(\frac{\Delta C}{S_C}\right)^2 + \left(\frac{\Delta H}{S_H}\right)^2 + R_T \times \frac{\Delta C}{S_C} \times \frac{\Delta H}{S_H}}$$

Compare that to $\Delta E_{76} = \sqrt{(\Delta L^2 + \Delta a^2 + \Delta b^2)}$. It's the same skeleton: three squared differences under a square root. Everything else is corrections.

What each correction is apologising for

Term	Formula	What it's fixing
S_L	$1 + 0.015(\bar{L}-50)^2 / \sqrt{20+(\bar{L}-50)^2}$	Lightness sensitivity dips at the extremes. The $(L-50)^2$ is a parabola centred on mid-grey — it's smallest in the middle, so corrections are smallest where we're sharpest.
S_C	$1 + 0.045 \times \bar{C}$	The more colorful the pair, the bigger a chroma gap must be before you notice. Grows straight-line with chroma.
S_H	$1 + 0.015 \times \bar{C} \times T$	Same idea for hue. T is a wobbly wave built from four cosines, because hue sensitivity varies depending on *which* hue you're at.
R_T	$-\sin(2\Delta\theta) \times R_C$	**The blue fix.** $\Delta\theta$ peaks at hue angle 275° — blue-violet. Away from blue this term is essentially zero; near blue it kicks in hard.

THE BAR OVER THE LETTER

$\bar{C}$ and $\bar{L}$ (read "C-bar", "L-bar") just mean the *average* of the two colors' values. The corrections are based on where the pair sits *on average*, not on either one alone — so the answer comes out the same whichever color you list first.

The catch you'll pay for later

Look at that last term again. It's a **cross-term**: chroma multiplied by hue, and it can be *negative*. That breaks something.

A proper distance (mathematicians say a **metric**) must obey the triangle inequality: going A→B→C can never be shorter than going straight A→C. It's the "no shortcut through a detour" rule, and it feels so obvious it's easy to forget it's an assumption.

ΔE_{00} **breaks it** in some regions. It is a *dissimilarity score*, not a true distance. Section 10 is entirely about the damage this does and how to work around it.

8 Finding the main colors: k-means

We can now measure color difference honestly. Next job: take a photo with two million pixels and boil it down to the handful of colors that actually define it.

The picture in your head

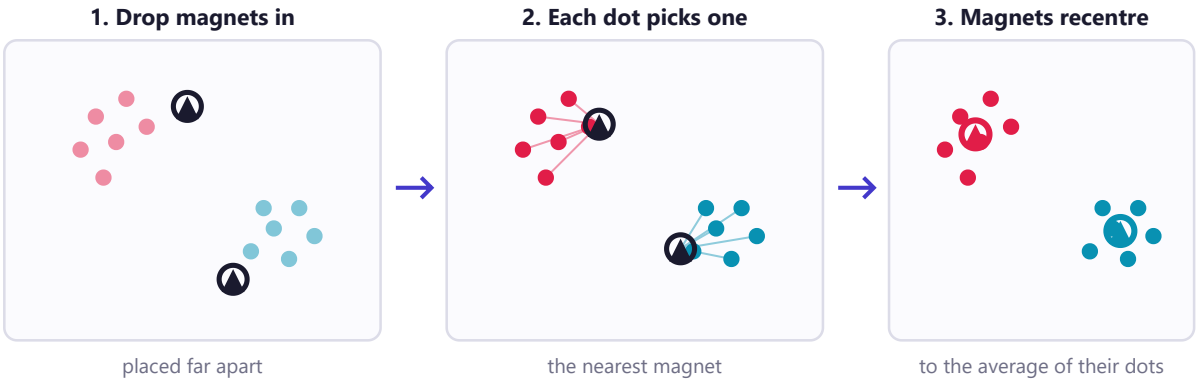
Take a photograph and throw away all arrangement — forget which pixel was where. You're left with a bag of two million colors. Now plot every one as a dot in the 3D Lab space from Section 5.

They will not spread out evenly. They form **clumps**. A beach photo makes a big dense cloud in the blue region (sky and sea), a smaller warm cloud (sand), a few scattered specks (a red umbrella). Your eye reads those clouds as "the colors of this photo."

So the task is precise: **find the centre of each cloud.** Those centres are the palette.

The algorithm, in four steps

This is **k-means**. The k is just how many colors you want. Imagine dropping k magnets into the dust cloud:



One round of k-means. Repeat steps 2 and 3 until nothing moves. The final magnet positions are your palette.

1. **Initialise.** Drop k magnets into the space. Not at random — use **k-means++**, which places the first one randomly and then deliberately places each next one *far* from all existing ones. Random placement can drop two magnets in the same cloud, leaving another cloud unclaimed, and the algorithm never recovers.
2. **Assign.** Every dot measures its distance to all k magnets and joins the closest one.
3. **Update.** Each magnet moves to the *average position* of all the dots that joined it.
4. **Repeat** steps 2 and 3. Dots switch teams, magnets shift again. Eventually nothing changes — the algorithm has **converged**. Done.

What it's actually minimising

Every algorithm like this is secretly trying to make one number as small as possible. Here that number is called **inertia** or WCSS (within-cluster sum of squares):

```
inertia = Σ (distance from each dot to its own magnet)²
```

Σ (capital sigma) means "add up all of these." So: for every dot, measure how far it is from its magnet, square that, and total it all. Low inertia means every dot is near its magnet — the palette represents the image well.

WHY *SQUARED* DISTANCE?

Two reasons. It kills minus signs (a dot 3 units left and one 3 units right both contribute 9, not +3 and −3 cancelling out). And it punishes big misses much harder than small ones — being 10 away costs 100, while being 1 away ten times costs only 10. That pressure is what pulls magnets toward genuine cloud centres.

But the real reason is deeper, and it's the subject of the next section.

9 Why the average is the best center — proved

Step 3 says "move to the average." Why the average? Why not the middle value, or something cleverer? This is provable in four lines, and the proof is the hinge the whole project turns on.

Let's simplify to one dimension — points on a number line. We have points x_1, x_2, …, x_n and we want to find the single position **c** that minimises the total squared distance to all of them:

$$f(c) = (x_1-c)^2 + (x_2-c)^2 + \dots + (x_n-c)^2$$

Step 1 — expand each bracket. Using $(a-b)^2 = a^2 - 2ab + b^2$:

$$(x_i - c)^2 = x_i^2 - 2x_i c + c^2$$

Step 2 — add them all up and group by powers of c:

$$f(c) = (\Sigma x_i^2) - 2c(\Sigma x_i) + n \cdot c^2$$

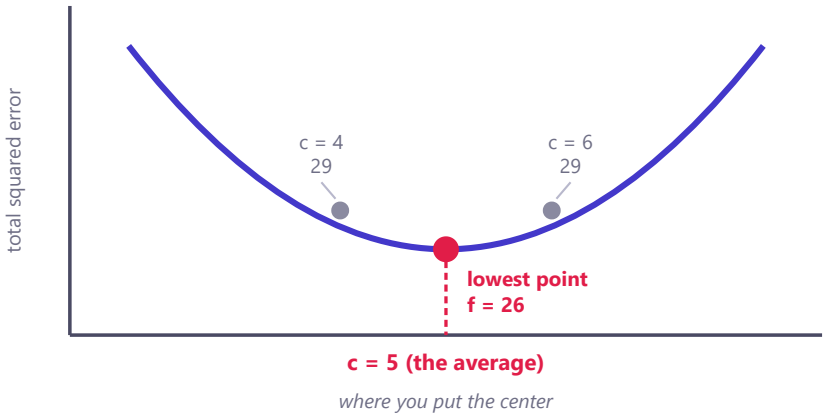
Step 3 — recognise the shape. Read that as a quadratic in c. Line it up against $ac^2 + bc + \text{constant}$:

```
a = n                (the number of points)
b = −2Σxᵢ           (minus twice the total)
constant = Σxᵢ²      (doesn't involve c, so it can't affect where the minimum is)
```

It's a **parabola**. And since a = n is a count of points, it's always positive — so the parabola opens upward and definitely has a lowest point.

Step 4 — find the vertex with the formula you already know, c = −b / 2a:

$$c = -(-2\Sigma x_i) \,/\, (2n) \;=\; 2\Sigma x_i \,/\, 2n \;=\; \Sigma x_i \,/\, n \;=\; \text{the average}$$



The proof, drawn. For points 2, 4 and 9 the error curve is a parabola whose vertex sits exactly at the average, 5. Note that 4 and 6 both score 29 — symmetric about the vertex, exactly as a parabola demands.

CHECK IT BY HAND

Points **2, 4, 9**. Average = (2+4+9)/3 = **5**.

`c = 5:` 9 + 1 + 16 = **26**

`c = 4:` 4 + 0 + 25 = 29 `c = 6:` 16 + 4 + 9 = 29

5 wins. And notice that 4 and 6 tie — they're equal distances either side of the vertex. Try any other value; you'll never beat 26.

WHY THIS MATTERS MORE THAN IT LOOKS

This is not a helpful rule of thumb. It's a **guarantee**. Step 3 of k-means doesn't just move the magnet somewhere better — it moves it to the *provably optimal* spot.

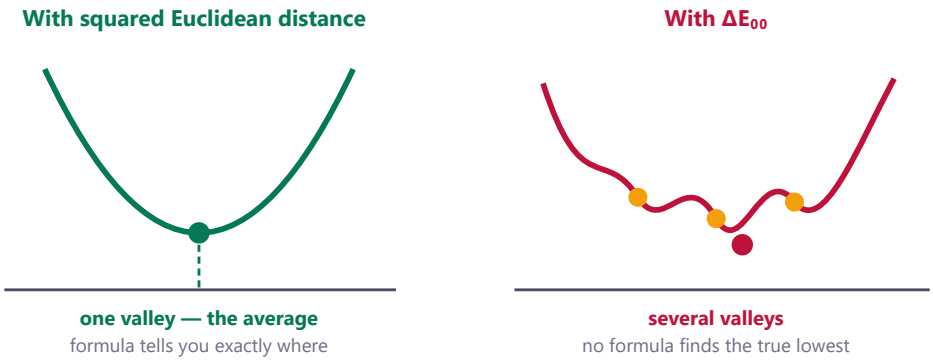
And that guarantee is what makes k-means work at all. Every round, the assign step can only lower inertia (dots move to closer magnets), and the update step can only lower inertia (magnets move to optimal positions). So inertia **strictly decreases every single round**. It can't go below zero. Therefore the algorithm must stop. It cannot loop forever.

Remember exactly how we got here: we expanded a **squared Euclidean** distance and a parabola fell out. That's the only reason any of it works.

10 Why ΔE_{00} breaks k-means — and the fix

Now the collision. We want ΔE_{00} because it's the honest ruler. But k-means only works because of the parabola. Put them together and something has to give.

Redo Section 9's proof with ΔE_{00} in place of ordinary distance. Step 1 fails immediately — there's nothing to expand. ΔE_{00} has square roots, sines, cosines, seventh powers, and that negative cross-term. Multiplying it out doesn't produce $ac^2 + bc +$ constant. It produces a lumpy curve with bumps and multiple valleys.



Left: the well-behaved parabola that makes k-means provably terminate. Right: what the error curve looks like under ΔE_{00}. The average is no longer guaranteed to be the bottom — it may not even be a bottom.

The consequences are concrete, not theoretical:

- The average is **no longer the best center**. Step 3 might move a magnet to a *worse* position.
- Inertia can therefore go **up** between rounds.
- The termination argument collapses. The algorithm can oscillate between two states forever.

The fix: stop calculating, start choosing

The trouble comes entirely from *calculating* a center — inventing a brand-new point in space via a formula that no longer applies. So don't calculate one.

Rule change: the center must be one of the actual data points.

Now step 3 becomes a search rather than a computation. For each cluster: try every member in turn as the candidate center, measure the total ΔE_{00} to all the others, and keep whichever scores lowest. A chosen center is called a **medoid**, and the algorithm is **k-medoids**.

WHY THIS RESCUES THE GUARANTEE

Because you literally try every option and keep the best one, the new center is *by construction* at least as good as the old one — the old center was one of the options you tried. Total error can never increase.

And this works for **any** measure of difference whatsoever. It doesn't need squares, or a parabola, or even the triangle inequality. You've traded a clever formula for honest brute force, and bought back the termination guarantee that ΔE_{00} destroyed.

The price, and how to pay it

Brute force costs. With n points in a cluster you test n candidates, and each test measures against n−1 others — roughly n^2 comparisons. At two million pixels that's 4×10^{12} ΔE_{00} evaluations per round. Completely impossible.

The fix is to shrink n before you ever start, using a **Lab histogram**:

```
Chop the Lab space into a 3D grid of small boxes (each 2 units on a side).
Sweep the image once. Each pixel drops into whichever box it lands in.
Keep only: box position + how many pixels landed there.
```

A typical UI screenshot has millions of pixels but only a few hundred to a few thousand *occupied* boxes. So n drops from 2,000,000 to about 2,000 — and because the cost goes as n^2, the work drops by a factor of about a **million**. Suddenly it runs in milliseconds.

BIN, DON'T SAMPLE

The obvious alternative is to grab 20,000 random pixels and cluster those. Don't. Random sampling throws information away, and it throws away exactly the wrong information.

Picture a website screenshot: white background, black text, and one small red logo covering 0.3% of the screen. That logo is the **brand color** — the single most important color on the page. Random sampling reduces it to about 60 stray pixels, and the clustering treats it as noise.

Histogram binning keeps *every* pixel's vote, with its correct weight. Nothing is lost, and n still comes down by a factor of a thousand. It's strictly better. It also gives you the cluster sizes for free, which you need anyway.

11 Contrast: can you actually read it?

We can now extract a beautiful palette from any image. But beautiful isn't enough — someone has to be able to *read* text painted in these colors.

Roughly 2.2 billion people live with some form of vision impairment. Low-contrast text is one of the most common and most easily avoided barriers on the web, and in many countries meeting a contrast standard is a legal requirement, not a nicety.

The current standard: WCAG 2.x

The Web Content Accessibility Guidelines define a **contrast ratio** between two colors using their luminance Y — which we already computed back in Section 4:

$$\text{contrast} = (Y_{\text{lighter}} + 0.05) \;/\; (Y_{\text{darker}} + 0.05)$$

The required minimums:

NORMAL TEXT	LARGE TEXT	ENHANCED
4.5 : 1	**3 : 1**	**7 : 1**
Level AA	18pt+, or 14pt bold	Level AAA

WHAT IS THAT + 0.05 DOING THERE?

Two jobs. Mathematically, pure black has Y = 0, and dividing by zero explodes. The 0.05 keeps the formula finite.

Physically, it models **screen glare**. A real screen showing pure black still reflects a bit of room light back at you — it's never truly black. The 0.05 is a stand-in for that reflected light.

Best possible score: white (Y=1) on black (Y=0) gives (1+0.05)/(0+0.05) = **21**. That's why the scale tops out at 21:1.

Two ways this formula is wrong

It's better than nothing, and it's the law in many places. But it has known, documented failures.

Flaw 1 — it can't tell light-on-dark from dark-on-light

The formula sorts the two colors into "lighter" and "darker" before dividing. So swapping text and background gives *exactly the same number*. Watch:

Grey `#767676` text on white — WCAG says **4.54 : 1** (passes AA)

White text on grey `#767676` — WCAG says **4.54 : 1** (identical!)

Same number. But they don't read the same, and the reason has a name: **halation**. Light text on a dark background appears to bleed and glow outward, blurring the letterforms. For readers with astigmatism —

and that's a large fraction of people — it's genuinely punishing. WCAG 2.x cannot see this at all, because it discards polarity before it starts computing.

Flaw 2 — it passes things that hurt to read

> Pure blue `#0000FF` on white scores **8.59 : 1** — sailing past even the strict AAA level of 7:1.

Now actually try to read a paragraph of that. Most people find it distinctly uncomfortable. Two reasons: blue contributes only 7.2% of luminance (Section 4), so the formula sees plenty of brightness difference; and the centre of your retina has very few blue-sensitive cones, so blue edges land soft and slightly out of focus.

The deeper issue is that WCAG 2.x's model assumes vision responds to light in a straight line. Section 3 showed that it doesn't — it's a curve. The formula also ignores font size and stroke weight completely, even though thin 12px text and bold 40px text obviously need different amounts of contrast.

12 APCA: the better contrast formula

APCA — the Advanced Perceptual Contrast Algorithm — is the proposed replacement, expected to land in WCAG 3.0.

Instead of a ratio, it produces a single number called $\mathbf{L^c}$ ("lightness contrast") on a scale from 0 to about 106.

The formula

Step 1 — screen luminance (note: a plain 2.4 power, no two-branch curve)

$$Y = 0.2127\times(R/255)^{2.4} + 0.7152\times(G/255)^{2.4} + 0.0722\times(B/255)^{2.4}$$

Step 2 — soft clamp for very dark colors

$$\text{if } Y < 0.022: \quad Y = Y + (0.022 - Y)^{1.414}$$

Step 3a — DARK text on LIGHT background

$$S = (Y_{bg}^{0.56} - Y_{text}^{0.57}) \times 1.14$$

$$L^c = (S - 0.027) \times 100$$

Step 3b — LIGHT text on DARK background

$$S = (Y_{bg}^{0.65} - Y_{text}^{0.62}) \times 1.14$$

$$L^c = (S + 0.027) \times 100$$

THE ONE THING TO NOTICE

Steps 3a and 3b are **different formulas** with different exponents. That is the entire point.

Look at 3a: the background is raised to 0.56 and the text to 0.57 — near-identical. Now 3b: 0.65 and 0.62 — a much wider gap, and both higher. APCA is saying that light-on-dark is a fundamentally different visual task, needing its own curve. That's the halation problem from Section 11, encoded directly into the math.

Notice also that text and background are *not* interchangeable — each has its own exponent. Swap them and you get a different answer, which is exactly right.

Reading the scale

Because APCA knows contrast depends on how big and how heavy the text is, the threshold isn't a single number — it's a table:

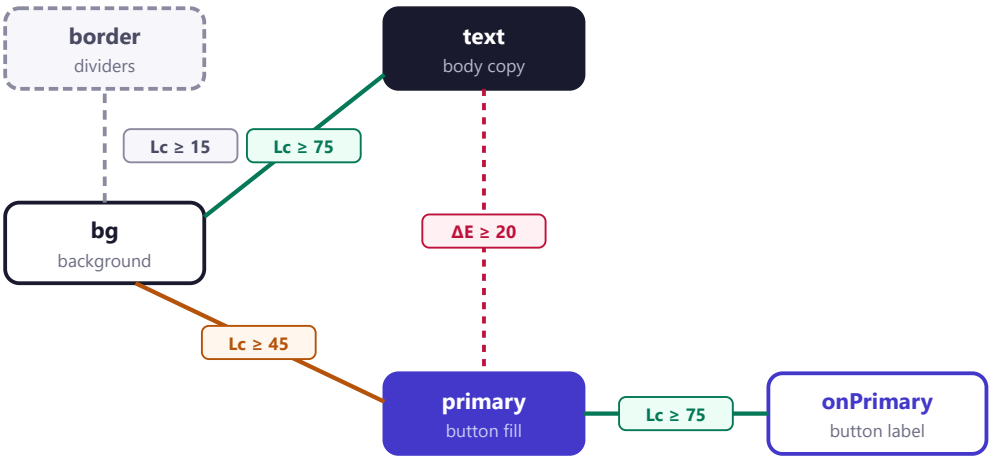
$\lvert L^c \rvert$	What it's good for
90	Preferred for body text; *required* for thin or small type
75	Minimum for body text (around 16px, normal weight)
60	Larger or heavier text (24px+, or 16px bold)
45	Large headings, 36px+
30	Absolute floor for any text — placeholders, disabled states
15	Non-text only: borders, dividers, decorative lines

One more detail: L^c comes out **signed**. Positive means dark-on-light, negative means light-on-dark. Use the size when comparing to the table, but keep the sign — it tells you which mode you're in.

13 The final puzzle: putting it together

Every piece is now on the table. Here's the problem they combine to solve — and why the obvious approach fails.

A real interface doesn't need "five nice colors." It needs colors assigned to **jobs**: a background, a card surface, body text, muted text, a primary button, the label on that button, a border, a danger color. And the requirements live in the *relationships between* those jobs, not in the colors themselves.



A simplified role graph. The boxes are jobs; the lines are hard requirements. Every single line must be satisfied at the same time, by one set of colors.

Why you can't just fix them one at a time

The tempting approach: generate a palette, check each rule, and nudge any color that fails.

It doesn't work, because **the rules are chained**. Suppose `text` is a little too light against `bg`. You darken it. Fine — but `text` also has to work against `surface`, and darkening it may have just broken that. Fix *that* and you might break the first one again. Real palettes get stuck in loops like this, and the loop isn't a bug in your patching logic — it's telling you the problem genuinely cannot be decomposed.

> **THE RIGHT WAY TO THINK ABOUT IT**
>
> This isn't a repair job. It's a **puzzle where every constraint must hold at once** — much closer to Sudoku than to spell-check. In Sudoku you don't fix one square at a time either; you reason about all the constraints together.
>
> Computer science has a name for this shape of problem — a **constraint satisfaction problem** — and solvers built for it that explore the possibilities systematically, ruling out whole branches at once.

The three-step plan

1. **Generate candidates.** Run k-medoids on the Lab histogram to get the image's main colors. Then expand each one into a ladder of lighter and darker versions, keeping its hue and colorfulness. Result: a few hundred candidates, all faithful to the original image.
2. **Precompute every pair.** Calculate L^c for all candidate pairs. A few hundred squared is under 100,000 calculations — a few milliseconds. Now every rule becomes a simple lookup: "which pairs are allowed for this line of the graph?"
3. **Solve the whole puzzle at once.** Hand the roles, the candidates and the allowed pairs to a constraint solver. It returns the combination that stays closest to the image's real colors while satisfying *every* rule — or it **proves that no combination exists**.

WHY "PROVES NONE EXISTS" IS A FEATURE, NOT A FAILURE

Sometimes the request is genuinely impossible. If someone insists on a pale yellow brand color for body text on a white background, no amount of cleverness will make that readable — the math simply doesn't allow it.

A guessing algorithm just fails quietly and hands back something bad. A solver tells you *which specific rule* is impossible, and why. That turns a dead end into useful advice: *"your brand yellow can't carry body text on white; here's the nearest shade that can."*

14 Cheat sheet

Everything, compressed. Keep this next to you while coding.

The conversion chain

```
sRGB (0–255)
   ↓  divide by 255
sRGB (0–1)
   ↓  C/12.92  if C ≤ 0.04045,  else ((C+0.055)/1.055)^2.4
linear RGB          ← real light amounts; do physics here
   ↓  three weighted sums (the 3×3 matrix)
XYZ                 ← what a standard eye measures; Y = luminance
   ↓  cube roots, then scale by 116 / 500 / 200
CIELAB (L*,a*,b*)   ← what the brain perceives; do geometry here
```

Key formulas

```
- linearize sRGB -
C_lin = C/12.92                    if C ≤ 0.04045
C_lin = ((C + 0.055)/1.055)^2.4    otherwise

- linear RGB to XYZ (D65) -
X = 0.4124 R + 0.3576 G + 0.1804 B
Y = 0.2127 R + 0.7152 G + 0.0722 B
Z = 0.0193 R + 0.1192 G + 0.9503 B

- XYZ to CIELAB -   white: Xn=0.9505  Yn=1.0000  Zn=1.0888
f(t) = t^(1/3)               if t > 0.008856
f(t) = (903.3 t + 16)/116    otherwise

L* = 116 f(Y/Yn) - 16
a* = 500 ( f(X/Xn) - f(Y/Yn) )
b* = 200 ( f(Y/Yn) - f(Z/Zn) )

- simple color difference (CIE76) -
dE76 = sqrt( dL^2 + da^2 + db^2 )

- corrected color difference (CIEDE2000) -
dE00 = sqrt( (dL/SL)^2 + (dC/SC)^2 + (dH/SH)^2 + RT (dC/SC)(dH/SH) )
  SL = 1 + 0.015 (Lbar-50)^2 / sqrt(20 + (Lbar-50)^2)
  SC = 1 + 0.045 Cbar
  SH = 1 + 0.015 Cbar T
  RT = -sin(2 dTheta) RC          (the blue correction)

- k-means objective -
inertia = SUM over all points of (distance to its own center)^2
best center under squared Euclidean = the average   (proved, Section 9)
best center under dE00 = search all members, keep the best  (k-medoids)

- WCAG 2.x contrast -
ratio = (Y_lighter + 0.05) / (Y_darker + 0.05)
AA: 4.5 normal text, 3.0 large text.  AAA: 7.0.  Max possible: 21.

- APCA -
Y = 0.2127 (R/255)^2.4 + 0.7152 (G/255)^2.4 + 0.0722 (B/255)^2.4
if Y < 0.022:  Y = Y + (0.022 - Y)^1.414
```

```
dark-on-light:  Lc = ((Ybg^0.56 - Ytxt^0.57) * 1.14 - 0.027) * 100
light-on-dark:  Lc = ((Ybg^0.65 - Ytxt^0.62) * 1.14 + 0.027) * 100
body text needs |Lc| ≥ 75.  Large headings ≥ 45.  Borders ≥ 15.
```

Numbers worth remembering

Number	Meaning
21.6%	The actual light output of RGB 128. Not 50%.
71.5%	Green's share of perceived brightness. Blue's is 7.2%.
0 – 100	The L* scale, black to white.
ΔE ≈ 1	Just-noticeable difference for a trained eye.
21 : 1	Maximum WCAG contrast (white on black).
L^c 75	APCA minimum for body text.
275°	The hue angle where ΔE_{00}'s blue correction peaks.

Glossary

Term	Meaning
sRGB	The standard color format of the web. Gamma-encoded, so the numbers aren't light amounts.
Gamma	The curve between stored number and emitted light. Exists so limited storage isn't wasted on brights.
Linearize	Undo gamma, recovering true light amounts. Required before any physics.
Luminance (Y)	How bright a color appears. Mostly green.
CIELAB / Lab	3D space where equal distances look equally different. L* = lightness, a* = green–red, b* = blue–yellow.
Chroma (C)	Colorfulness — how far from grey. Distance from the centre line in Lab.
Hue (H)	Which color it is — the angle around the centre line.
ΔE	A number for "how different do these two colors look." 1 ≈ just noticeable.
ΔE_{00} / CIEDE2000	The accurate but complicated version, with corrections for chroma, lightness and blue.
Perceptually uniform	Equal number steps produce equal visual steps. RGB isn't; Lab nearly is.
Centroid	A cluster center that is *calculated* (the average). Used by k-means.
Medoid	A cluster center that is *chosen* from real data points. Used by k-medoids.
Inertia / WCSS	Total squared distance from every point to its own center. Clustering minimises this.
Convergence	The point where more rounds change nothing. The algorithm stops.
Triangle inequality	A detour is never shorter than going direct. True distances obey it; ΔE_{00} doesn't.
WCAG	Web Content Accessibility Guidelines. Version 2.x defines the current contrast-ratio rule.
APCA / L^c	The proposed replacement contrast metric. Polarity-aware, on a 0–106 scale.
Halation	The optical glow of light text on dark backgrounds. Painful with astigmatism; invisible to WCAG 2.x.
Polarity	Which way round it is: dark-on-light or light-on-dark. Different visual tasks.
Gamut	The set of colors a device can physically display. Lab can describe colors your screen can't show.

THE WHOLE DOCUMENT IN SIX LINES

1. A color is a point in 3D space.

2. RGB is a distorted map of that space, so we convert to Lab, where distance means something.

3. Distance in Lab is Pythagoras; ΔE_{00} is Pythagoras with corrections bolted on.

4. A palette is the set of cloud-centres in that space, found by k-means — or k-medoids, once ΔE_{00} breaks the average.

5. Contrast decides whether text is readable; WCAG 2.x gets it wrong in two known ways, and APCA fixes both.

6. Real palettes must satisfy every contrast rule *simultaneously*, which makes it a puzzle for a constraint solver, not a series of patches.